

OpenFrame IPF管理者ガイド

OpenFrame/Batch for MVS 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame IPF管理者ガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/Batch for MVS 7.1

ガイドバージョン: v3.1.3

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 IPFの紹介	1
1.1. 概要	1
1.2. IPFの機能	1
1.3. IPFの構造	2
1.4. IPFの接続と終了	3
1.4.1. 接続および実行	3
1.4.2. 終了	4
第2章 IPFのサブルーチン	5
2.1. 概要	5
2.2. IPFCMD	6
2.3. IPFCVS	6
2.4. IPFMIO	7
2.5. IPFMRR	8
2.6. IPFMTO	9
2.7. IPFOVS/IPFOPN	9
2.8. IPFPFF/IPFPFS	10
2.9. IPFRIO	11
第3章 メニュー定義体	13
3.1. メニュー定義体の特徴	13
3.2. メニュー定義体の構成	13
3.2.1. 属性定義部	13
3.2.2. メニュー原型部	15
3.2.3. アクション定義部	16
3.3. tsomapgen	18
第4章 画面処理の応用	21
4.1. LSDA	21
4.2. 画面分割	22
4.2.1. 複数の論理画面	22
4.2.2. 分割された画面の識別	22
4.3. 画面移動(スクロール)	22
4.3.1. 画面移動機能	22
4.3.2. 画面移動のファンクション・キー	23
4.4. 画面固定	23

図目次

[図 1.1] IPFの構造	2
----------------------	---

このガイドについて

対象読者

本書は、リホスト・ソリューションであるTmax OpenFrame[®](以下、OpenFrame)で使用されるIPF(Interactive Programming Facility)を使用および管理するユーザーを対象としています。

IPFを使用してCOBOLアプリケーション・プログラムなどからTSOコマンドを呼び出すか、スクリーン型のディスプレイ端末への入出力を行うことができます。

前提知識

本書を理解するには、IPF、TSO、CLISTに関する知識が必要です。

制限事項

本書では、IPFで現在サポートしていない機能、オプション、パラメータは別途表記しており、説明は省略しています。また、OpenFrame GWの詳細機能については、OpenFrame GWのマニュアルを参照してください。

本書の構成

本書は、計4章で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: IPFの紹介

IPFの概念、機能および構造について説明します。

- 第2章: IPFのサブルーチン

IPFの機能を実行するためのサブルーチンとパラメータについて説明します。

- 第3章: メニュー定義体

画面入出力処理のためのマップ・ファイルであるメニュー定義体およびマップ・ファイルのコンパイル・ツールであるtsomapgenの使用方法について説明します。

- 第4章: 画面処理の応用

画面分割、画面移動、画面固定など、画面処理の応用動作について説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上 (32ビット, 64ビット)
	Linux x86 2.6以上 (32ビット, 64ビット)
	Linux ia64 2.6以上 (32ビット, 64ビット)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 (Fixset07)
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame TJESガイド	TJESを使用してOpenFrameシステムのバッチ・ジョブを管理する方法について説明しています。
OpenFrame ユティリティ リファレンスガイド	OpenFrameエンジンと一緒に提供される多様なユティリティ・プログラムについて説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

第1章 IPFの紹介

本章では、IPFの概念、機能および構造について説明します。

1.1. 概要

IPF(Interactive Programming Facilities)は、COBOLなどのアプリケーション・プログラムからTSO(Time Sharing Option)コマンドを呼び出すか、スクリーン型のディスプレイ端末に入出力することができるプログラムです。TSOの詳細については、『OpenFrame TSO管理者ガイド』を参照してください。

コンピューター・システムの利用分野が拡大されるにつれ、エンドユーザーからも操作しやすいシステムへの要求がますます大きくなっています。データベースをベースにしてデータの操作や検索などをより簡単かつ効率的に行い、その結果を使用環境のディスプレイ端末に出力して表示する機能が必要とされています。

TSOコマンドを実行する際にも、ユーザー利便性のためにディスプレイ端末を通じた入出力処理が要求されます。IPFは、これらのニーズに応じて開発されたユーザーとシステム間のディスプレイ端末を介した対話型プログラミング・パッケージを提供します。

1.2. IPFの機能

以下は、IPFが提供する機能です。

- TSOコマンド処理

IPFでは、COBOL、PL/Iなどの高級言語で作成されたプログラムからTSOコマンドまたはCLISTを呼び出して実行できる機能を提供します。この機能を利用するためには、IPFCMDサブルーチンのパラメータにTSOコマンドまたはCLIST名を記述した文字列を指定してCALL文として呼び出す必要があります。IPFCMDの詳細については、「[第2章 IPFのサブルーチン](#)」を参照してください。

- フルスクリーン型の入出力処理

IPFでは、ディスプレイ端末に対して高級言語においても画面単位でのデータ入出力処理が簡単に実行できる機能を提供します。フルスクリーン型ディスプレイ端末では、表示画面全体をフィールドという単位の集まりと見なします。フィールドは、データの入出力処理を実行する最小限の単位であり、これはまた、テキスト・フィールド、入力フィールド、出力フィールドに分けることができます。すなわち、フルスクリーン型ディスプレイ端末とは、フィールドの集まりに対して入出力処理を同時に行えるディスプレイ端末を意味します。

フルスクリーン型ディスプレイ端末の標準的な画面サイズは、横80×縦24です。固定されたサイズのディスプレイ端末に関係なく、ユーザーはより大きな画面を設計することができますが、これを**論理画面**といい、ディスプレイ端末を**物理画面**といいます。

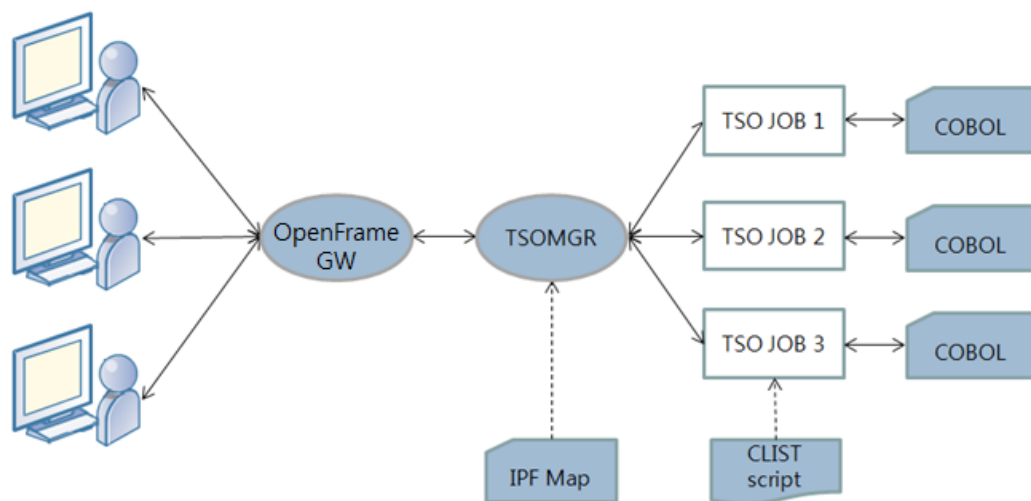
論理画面は、メニュー定義体と呼ぶマップ・ファイルに定義することが可能であり、IPFMIOのようなIPFのサブルーチンを利用して論理画面を物理画面に表示することができます。IPFMIOとメニュー定義体の詳細については、「[第2章 IPFのサブルーチン](#)」と「[第3章 メニュー定義体](#)」を参照してください。

1.3. IPFの構造

IPFとは、高級言語(COBOL、PL/Iなど)のサブルーチンの集まりです。アプリケーション・プログラムは、このIPFサブルーチン呼び出すことで目的の機能を実行することができます。IPFサブルーチンでは、アプリケーションの呼び出しを受けると、TSOMGRのTmaxサービス呼び出してデータとパラメータを渡します。画面の入出力と関連したIPFサブルーチンの場合、TSOMGRはコンパイル済みのマップ情報ファイルを読み込んで、画面のフィールド情報を構成し、OpenFrame GWを介してディスプレイ端末に出力します。画面に入力値が存在する場合は、TSOMGRがOpenFrame GWを通じてその値を取得し、入力データを抽出してから再びIPFサブルーチンを通じてアプリケーション・プログラムに返します。

以下の図は、IPFシステムの構造を示しています。

[図 1.1] IPFの構造



- TSOMGR

アプリケーション・プログラムで呼び出したIPFサブルーチンからデータとパラメータを受け取り、コンパイル済みのマップ情報ファイルから必要なマップ情報を読み込んで、画面のフィールド情報を構成してからOpenFrame GWに渡します。画面に入力値がある場合は、OpenFrame GWを介してその値を取得し、必要なデータを抽出してからIPFサブルーチンによりアプリケーション・プログラムに返します。

- IKJEFT01

ユーザーがディスプレイ端末を介してIPFに接続すると、TSOMGRがそれぞれのTSOユーザーに対するジョブを実行させます。それぞれのジョブはIKJEFT01ユティリティを利用して、指定されたCLISTスクリプトを分析してTSOコマンドを実行させます。TSOのCALLコマンドを使用してCOBOLなどのアプリケーション・プログラムを呼び出すことができます。

- IPFサブルーチン

IPFサブルーチンは、COBOLのようなアプリケーション・プログラムに記述され、TSOMGRのTmaxサービスを呼び出します。IPFCMDサブルーチンはTSOコマンドを呼び出す機能を実行し、IPFMIOサブルーチンは画面への入出力を処理する機能を実行します。IPFのそれぞれのサブルーチンを呼び出すときや、アプリケーション・プログラムでIPFサブルーチンの情報をやり取りするときは、CALL文のパラメータを使用して呼び出します。

- OpenFrame GW

OpenFrameが提供するWeb端末などのディスプレイ端末でユーザーが入力したデータをTSOMGRに渡し、アプリケーション・プログラムで処理したデータをTSOMGRを介して受け取ってディスプレイ画面に出力します。

- TSOMAPGEN

ユーザーが作成したマップ・ファイルを読み込んで解析(Parsing)し、.mapファイル形式にコンパイルして指定したライブラリに保存します。

1.4. IPFの接続と終了

IPFを実行するために、3270端末またはOpenFrameが提供するWeb端末などのディスプレイ端末に接続してログインします。ディスプレイ端末に接続した後、**TSO**コマンドを入力すると、ログイン画面が表示されます。ログイン情報を入力した後に準備画面が表示されたら、TSOコマンドまたはCLIST名を入力して、IPFを実行することができます。すべての作業が終わったら、ログオフしてIPFを終了します。

1.4.1. 接続および実行

ディスプレイ端末に接続すると、カーソルが位置する入力画面が表示されます。そのとき、以下のようにIPFを起動するためのTSOコマンドを入力します。

TSO

上記のコマンドを入力して<Enter>キーを押すと、ログイン画面が表示されます。

```
-----  
USERID      :  
PASSWORD    :  
PROCEDURE   :  
-----
```

ユーザーID、パスワード、プロシージャー名を入力して<Enter>キーを押すと、準備画面が表示されます。

```
READY
```

READYの下部にカーソルが位置した入カスペースが表示されます。この部分にTSOコマンドまたは実行するCLIST名を入力して目的の動作を実行することができます。

1.4.2. 終了

TSOまたはCLISTコマンドを実行すると、終了メッセージが表示されます。そのとき<Enter>キーを押すと画面がクリアされ、準備画面(READY)の状態に戻ります。終了する場合は、入力画面にLOGOFFコマンドを入力します。

```
READY  
LOGOFF
```

LOGOFFコマンドを実行すると、以下のとおりログオフ画面を呼び出してIPFを終了します。

```
-----  
SESSION LOGOFF  
-----
```


第2章 IPFのサブルーチン

本章では、IPFサブルーチンの動作およびアプリケーション・プログラムとのインターフェース形式について説明します。

2.1. 概要

IPFには、アプリケーション・プログラムからTSOコマンドを呼び出す機能とアプリケーション・プログラムから画面にデータの入出力を処理する機能があります。

以下は、OpenFrameが提供するIPFのサブルーチンです。

サブルーチン	説明
IPFCMD	アプリケーション・プログラムからTSOコマンドを呼び出します。
IPFCVS	論理画面を終了します。
IPFMIO	メニュー定義体を利用して画面の入出力処理を行います。
IPFMRR	IPFMIO/IPFMTOが呼び出された後、メッセージを出力します。日本語に対応します。
IPFMTO	メニュー定義体を利用して画面の出力処理を行います。
IPFOVS/IPFOPN	論理画面を開きます。
IPFPFF/IPFPFS	画面移動のために予約されているファンクション・キーを解除または再設定します。
IPFRIO	IPFMIO/IPFMTOが呼び出された後、同一画面に対して更新(Refresh)機能を行います。

アプリケーション・プログラムとのインターフェース

COBOLなどのアプリケーション・プログラムからCALLコマンドを使ってIPFサブルーチンを呼び出してデータを送受信することができます。

以下は、COBOLでIPFサブルーチンを呼び出す例です。

```
CALL 'IPFCMD' USING RC1 RC2 CMD L.  
...  
CALL 'IPFOVS' USING PIA LSIA RCA.  
...  
CALL 'IPFMIO' USING LSIA MIA MDA RCA.
```

```
...  
CALL 'IPFCVS' USING LSIA RCA.
```

2.2. IPFCMD

アプリケーション・プログラムからTSOコマンドを呼び出します。

- 構文

```
CALL IPFCMD  
    RC1  
    RC2  
    CMD  
    L
```

以下は、IPFCMDのパラメータについての説明です。

項目	説明
RC1	IPFCMDのリターンコードを設定する4バイトの領域です。
RC2	TSOコマンドのリターンコードを設定する4バイトの領域です。
CMD	TSOコマンドを記述するLバイトの文字領域です。
L	CMDの長さを設定する領域です。(最大長: 32760)

2.3. IPFCVS

論理画面を終了します。同サブルーチンが呼び出された後には、画面入出力はできません。

- 構文

```
CALL IPFCVS  
    LSIA  
    RCA
```

以下は、IPFCVSのパラメータについての説明です。

項目	説明
LSIA	終了する論理画面を識別ための128バイトの領域です。 IPFOVS/IPFOPNで開くときに指定したLSIAと同じである必要があります。
RCA	IPFCVSを実行した後、リターンコードが設定される4バイトの領域です。

2.4. IPFMIO

区分データセット(PDS)にメンバーとして登録されたメニュー定義体を利用して、画面の入出力処理を行います。

- 構文

```
CALL IPFMIO  
      LSIA  
      MIA  
      MDA  
      [N PMA]  
      [CSA]  
      RCA
```

以下は、IPFMIOのパラメータについての説明です。

項目	説明
LSIA	アクセスする論議画面を識別するための128バイトの領域です。 LSIAを有する論理画面が先に開かれている必要があります。
MIA	メニューを識別するための16バイトの領域です。 DD名の8バイトとメンバー名の8バイトで区分されます。 – DD名：使用するメニュー定義体をメンバーとして有するデータセットのDD名を指定します。 – メンバー名：使用するメニュー定義体のメンバー名を指定します。
MDA	メニュー定義体のパラメータ指定に対応して実際に転送される入出力データが保存される領域です。
N	属性が変更されるパラメータ領域(PMA)の数です。ゼロの場合、PMAの値は無視されます。(PMAと一緒に省略可能)
PMA	パラメータ・フィールドの属性を一時的に変更するために使用される4×Nバイトの領域です。 以下のように3つの領域で構成されます。(Nと一緒に省略可能) – パラメータ番号(PNO): 変更するアクション・パラメータ番号を指定する2バイトの領域です。 – 属性文字(ATTR)：新しく与える属性を指定する1バイトの領域です。 – 変更通知文字(M)：OpenFrameではサポートしていません。
CSA	カーソルの位置を設定する4バイトの領域です。 以下のように2つの領域で構成されます。(省略可能) – CSA1：カーソルを位置させるパラメータ番号を2バイトで設定します。

項目	説明
	– CSA2: 指定したパラメータ内の先頭からの位置を2バイトで設定します。
RCA	IPFMIOを実行した後、リターンコードが設定される4バイトの領域です。

2.5. IPFMRR

IPFMIO/IPFMTOが呼び出された後、メッセージを出力するために使用されます。日本語のメッセージも出力可能です。

● 構文

```
CALL IPFMRR
      LSIA
      MA
      MPA
      MDA
      [PNA1 PNA2 ...]
      [CSA]
      RCA
```

以下は、IPFMRRのパラメータについての説明です。

項目	説明
LSIA	アクセスする論理画面を識別するための128バイトの領域です。 LSIAを有する論理画面が先に開かれており、IPFMIO/IPFMTOで指定されたLSIAと同じである必要があります。
MA	メッセージとして出力するデータを含んでいる領域です。日本語のデータも使用可能です。
MPA	メッセージを出力する論理画面上の行の位置を指定します。
MDA	メニュー定義体のパラメータ指定に対応して実際に転送される出力データが保存される領域です。
PNAi	属性を変更するパラメータ番号を指定する2バイトの領域です。 OpenFrameでは、エラーが発生しないように解析(Parsing)のみサポートしています。(省略可能)
CSA	カーソルの位置を設定する4バイトの領域です。詳細については、 「2.4. IPFMIO」 を参照してください。
RCA	IPFMRRを実行した後、リターンコードが設定される4バイトの領域です。

2.6. IPFMTO

区分データセット(PDS)にメンバーとして登録されたメニュー定義体を利用して画面上に出力処理のみを行います。IPFMIOとは違って入力値を受けず、画面を出力した後、すぐに返されます。

- 構文

```
CALL IPFMTO  
      LSIA  
      MIA  
      MDA  
      [N PMA]  
      [CSA]  
      RCA
```

以下は、IPFMTOのパラメータについての説明です。

項目	説明
LSIA	アクセスする論理画面を識別するための128バイトの領域です。 LSIAを有する論理画面が先に開かれている必要があります。
MIA	メニューを識別するための16バイトの領域です。 DD名の8バイトとメンバー名のバイトで区分されます。 – DD名：使用するメニュー定義体をメンバーとして有するデータセットのDD名を指定します。 – メンバー名：使用するメニュー定義体のメンバー名を指定します。
MDA	メニュー定義体のパラメータ指定に対応して実際に転送される出力データが保存される領域です。
N	属性が変更されるパラメータ領域(PMA)の数です。 ゼロの場合、PMAの値は無視されます。(PMAと一緒に省略可能)
PMA	パラメータ・フィールドの属性を一時的に変更するために使用される4×Nバイトの領域です。詳細については、 「2.4. IPFMIO」 を参照してください。
CSA	カーソルの位置を設定する4バイトの領域です。詳細については、 「2.4. IPFMIO」 を参照してください。
RCA	IPFMTOを実行した後、リターンコードが設定される4バイトの領域です。

2.7. IPFOVS/IPFOPN

論理画面を開きます。画面の入出力処理のためには、先に同サブルーチンを呼び出す必要があります。

- 構文

```
CALL IPFOVS | IPFOPN  
    PIA  
    LSIA  
    [LSDA]  
    RCA
```

以下は、IPFOVS/IPFOPNのパラメータについての説明です。

項目	説明
PIA	プログラムを識別するための4バイトの領域です。
LSIA	論理画面を識別するための128バイトの領域です。
LSDA	論理画面の定義領域です。論理画面の属性を指定する情報が含まれている24バイトの領域です。(省略可能)
RCA	IPFOVS/IPFOPNを実行した後、リターンコードが設定される4バイトの領域です。

2.8. IPFPFF/IPFPFS

IPFでデフォルト値として指定されているスクロール・キー(PF7、PF8、PF10、PF11、PF19、PF20、PF22、PF23)を解除(IPFPFF)するか、解除されたキーを再設定(IPFPFS)する機能です。

- 構文

```
CALL IPFPFF | IPFPFS  
    LSIA  
    [PFN1 PFN2 ...]  
    RCA
```

以下は、IPFPFF/IPFPFSのパラメータについての説明です。

項目	説明
LSIA	アクセスする論理画面を識別するための128バイトの領域です。このLSIAを有する論理画面が先に開かれている必要があります。
PFNi	スクロール・キーを解除または再設定する場合、該当するPFキー番号を指定します。 指定できる番号は以下のとおりです。 – PF7またはPF19：上方向スクロール・キー – PF8またはPF20：下方向スクロール・キー – PF10またはPF22：右方向スクロール・キー

項目	説明
	– PF11またはPF23 : 左方向スクロール・キー PFNiは省略可能であり、省略した場合は8つのスクロール・キーをすべて指定したのと同じ動作をします。
RCA	IPFPFF/IPFPFSを実行した後、リターンコードが設定される4バイトの領域です。

2.9. IPFRIO

IPFMIO/IPFMTOが呼び出された後、同じ画面の入出力を行う場合に使用します。同一画面に対する入出力処理であるため、IPFMIO/IPFMTOを使用するよりも効率的です。

● 構文

```
CALL IPFRIO
      LSIA
      MDA
      [N PMA]
      [CSA]
      RCA
```

以下は、IPFRIOのパラメータについての説明です。

項目	説明
LSIA	アクセスする論理画面を識別するための128バイトの領域です。 LSIAを有する論理画面が先に開かれており、IPFMIO/IPFMTOで指定されたLSIAと同じである必要があります。
MDA	メニュー定義体のパラメータ指定に対応して実際に転送される出力データが保存される領域です。
N	属性が変更されるパラメータ領域(PMA)の数です。ゼロの場合、PMAの値は無視されます。(PMAと一緒に省略可能)
PMA	パラメータ・フィールドの属性を一時的に変更するために使用される4×Nバイトの領域です。詳細については、 「2.4. IPFMIO」 を参照してください。
CSA	カーソルの位置を設定する4バイトの領域です。詳細については、 「2.4. IPFMIO」 を参照してください。
RCA	IPFRIOを実行した後、リターンコードが設定される4バイトの領域です。

第3章 メニュー定義体

本章では、メニュー定義体の特徴およびメニュー定義体を利用した論理画面へのアクセスについて説明します。

3.1. メニュー定義体の特徴

メニュー定義体は区分データセット(PDS)のメンバーであり、論理画面上のフィールドを視覚的に定義することができます。

ユーザーが作成するメニュー定義体はテキスト・ファイル形式のドキュメントです。OpenFrameでは、[tsomapgen](#)というコンパイル・ツールを使用して、*menuname.map*(コンパイルするメニュー定義体の名前.map)形式のバイナリ・ファイルにコンパイルおよび保存します。以降、IPFプログラムでそのファイルから必要な情報を読み込んで使用します。

3.2. メニュー定義体の構成

3.2.1. 属性定義部

属性(attribute)定義部は、属性文字を変更または追加するか、属性文字に別の属性を追加する場合に使用します。標準属性文字のみを使用する場合には、省略することが可能です。属性定義部は、<ATTR>で始まり<END>で終わります。

以下は、形式で使用する標準属性文字と属性記号についての説明です。

- 標準属性文字

項目	説明
^	入力フィールドの属性を持ちます。表示されないフィールドです。
%	入力フィールドの属性を持ちます。明るく表示されます。
	入力フィールドの属性を持ちます。暗く表示されます。
&	テキスト・フィールドの属性を持ちます。明るく表示されます。
¥ (通貨記号)	テキスト・フィールドの属性を持ちます。暗く表示されます。
#	出力フィールドの属性を持ちます。明るく表示されます。
@	出力フィールドの属性を持ちます。暗く表示されます。

- 属性記号

項目	説明
A	更新(Refresh)するとき、データを保持します。
B	点滅します。
D	表示されないフィールドです。
E	更新するとき、データを削除します。
H	明るく表示されるフィールドです。
I	入力フィールドです。
J	右揃えされるフィールドです。
N	数字のみ使用可能なフィールドです。
O	フィールドにオーバーラインを表示します。
P	出力のみ可能なフィールドです。
R	フィールド内のデータは暗く、それ以外のフィールド領域は明るく表示します。
U	フィールドにアンダーラインを表示します。
V	フィールドにバーティカルラインを表示します。

属性定義部は、以下の形式で設定することができます。それぞれの形式を複数行にわたって一緒に使用することが可能です。

- 形式1

- 構文

任意の文字 = 標準属性文字 [+ 属性記号];

右辺に記述された標準属性文字の属性を、左辺に記述された任意の属性文字に移します。必要に応じて別の属性を追加することもできます。属性が移された標準属性文字の既存の属性は消えます。以降、メニュー原型部では一般的なデータ用の文字として使用することができます。

- 例

```
<ATTR>
X = % + OUV;
<END>
```

Xという文字を新しい属性文字として定義します。標準属性文字である%の属性をXに与え、Over line(O)、Under line(U)、Vertical line(V)などの属性を追加します。以降、%は属性文字として使用できません。

- 形式2

- 構文

```
標準属性文字 =+ 属性記号;
```

右辺に記述された属性記号の属性を、左辺に記述された標準属性文字に追加します。

- 例

```
<ATTR>  
@ =+ JN;  
<END>
```

標準属性文字である@の右辺に属性(J)と数字フィールド属性(N)を追加します。

- 形式3

- 構文

```
任意の文字 =+ 属性記号;
```

右辺に記述された属性記号の属性を、左辺に記述された任意の属性文字に追加します。そのとき使用された任意の文字は、属性文字として使用できるため、以降、メニュー原型部では一般的なデータ用の文字として使用することはできません。

- 例

```
<ATTR>  
Y =+ IOUV;  
<END>
```

Yという新しい属性文字を定義し、入力フィールド(I)、Over line(O)、Under line(U)、Vertical line(V)などの属性を与えます。

3.2.2. メニュー原型部

メニュー原型部は、属性文字、テキスト・データ、パラメータ名(フィールド名)で構成されており、画面に実際に出力される領域です。属性文字は1バイトの領域を占めており、フィールドの段落および後述のフィールドの属性を定義します。

テキスト・データは、そのままメニュー原型部に記述することが可能であり、画面の同じ位置に出力されます。データの入力または出力のためのフィールドには、それぞれのパラメータ名(フィールド名)を記述します。アプリケーション・プログラムと画面との間でデータを送受信する役割をします。入出力フィールドについては、後述のアクション定義部にてより詳しく定義します。

以下は、メニュー原型部の特性です。

- メニュー原型部はフィールドの集まりです。
- 1行に複数のフィールドがつながることがありますが、1つのフィールドが複数行にわたってつながることはできません。

- 1つのフィールドは、1つの属性文字とその次の属性文字の間の文字列です。
- フィールド内の文字列は、テキスト・データであるか、パラメータ名あるいはフィールド名、そしてそれに伴う空白列で構成されます。
- 表示データは1バイトの文字列であるか、2バイト(日本語)の文字列、または1バイトと2バイトが混用された文字列です。
- こちらでのパラメータ名とは、アクション定義部で記述されるパラメータ名を意味しており、フィールド名は、アクション定義部のFIELDオペランドに記述されるフィールド名を意味します。

3.2.3. アクション定義部

アクション定義部では、メニュー原型部で指定された入出力フィールドのパラメータについて詳しい属性を定義します。入出力フィールドのパラメータは、アプリケーション・プログラムで指定されたMDAデータを利用して取得し、使用しないパラメータは省略することが可能です。アクション定義部は<ACTION>で始まり<END>で終わります。

アクション定義部は以下のように構成されます。

```
パラメータ番号 {パラメータ名*}
                {CHAR(n) | NCHAR(n) | FIXED(n) | DEC(n[,pos])}
                [LIST(d1) RETURN(d2)]
                [INIT(初期値 | \PARMn)]
                [FIELD(フィールド名)]
                [CURSOR]
                [IMAGE]
                [/* コメント */]
```

以下は、アクション定義部の各オペランドについての説明です。

項目	説明
パラメータ番号	メニュー定義体のすべてのパラメータに対して付与される番号であり、1から始まり連続して続きます。アクション・フィールドのパラメータ番号の順序は、アプリケーション・プログラムから渡されるデータの順序であり、メニュー定義体の順序とは関係ありません。
パラメータ名	メニュー定義体のパラメータに与える名前です。メニュー原型部の同じ名前を持つフィールドとマッチングされます。
*	MDAとデータが対応するとき、入出力の対象として扱わないデータをアステリスク(*)で宣言します。
CHAR(n)	nバイトの文字列フィールドを意味します。
NCHAR(n)	日本語データ(2バイト)のフィールドを意味します。このときの長さ(n)は、日本語の文字数であり、実際の長さは2×nバイトです。

項目	説明
FIXED(n)	符号付きまたは符号無し整数型フィールドを意味します。 桁数(n)は15、16、31、32に限定され、15、31は符号付き整数、16、32は符号無し整数です。
DEC(n[,pos])	符号付き10進数の固定小数点データです。桁数(n)は15、7に限定されます。 小数点指定引数(pos)は、データの右端からnの分だけの小数点位置です。省略する場合はゼロになります。
LIST(d1) RETURN(d2)	LISTに指定されたd1が入力されたら、RETURNに指定されたd2をアプリケーション・プログラムに返します。
INIT	表示するデータが存在しない場合は、以下の設定値でデータを初期化します。 - 初期値：初期値に指定された値でデータが初期化されます。 - PARMn：MDAのn番目のパラメータの値でデータが初期化されます。アプリケーション・プログラムで出力データを表示するためには、同オプションを設定する必要があります。省略する場合は、空白が表示されます。
FIELD	メニュー原型部上にパラメータ名ではなく、同オペランドで指定されたフィールド名が明示されていることを示します。OpenFrameでは解析(Parsing)のみサポートしています。
CURSOR	カーソルを設定するフィールドを示します。同オペランドが省略された場合は、最初の入力フィールドにカーソルを位置させます。
IMAGE	FIXEDまたはDECで定義されたパラメータの数値データを画面上に表現する形式を以下のような文字列で指定することができます。 +：最初の位置に表示された場合は正のデータです。有効数字に先行するゼロは、すべてゼロサプレスします。 -：最初の位置に表示された場合は負のデータです。有効数字に先行するゼロは、すべてゼロサプレスします。 ¥：通貨記号です。有効数字の先頭に「¥」を表示します。これに先行するゼロは、すべてゼロサプレスします。 _：ゼロサプレス文字です。有効数字に先行するゼロは、すべてゼロサプレスします。 *：ゼロサプレス文字です。有効数字に先行するゼロは、すべて「*」に置き換えます。 #：ゼロサプレスせずに、数字を示します。

項目	説明
	– 点(.) : 小数点の位置を示します。 – コンマ(,) : 挿入文字として使用します。ゼロサプレスが発生する場合を除いて、対応する位置に「,」が挿入されます。
コメント	アクション定義部にコメントを記述することができます。 コメント文は「/*」と「*/」の間に記述しますが、その内容に「/*」または「*/」を含むことはできません。

3.3. tsomapgen

tsomapgenは、テキスト・ファイルで作成された最初のメニュー定義体ファイルを解析およびコンパイルして、バイナリ・ファイル形式で保存するツールです。TSOMGRやIPFサブルーチンでメニュー定義体を参照する場合、コンパイルされたマップ・ファイルを読み込んで情報を取得します。

• 構文

```
usage : tsomapgen [options] menuname
```

– [options]

オプション	説明
-m <i>program</i>	コンパイルするときの必須オプションであり、コンパイルするメニュー定義体の種類を指定します。 tsomapgenでコンパイルできるメニュー定義体には、IPFマップがあります。
[-l <i>pdsname</i>]	コンパイルされたマップ・ファイルを保存する場所を指定します。 マップ・ファイルは、区分データセット(PDS)のメンバーとして保存されます。同オプションを省略すると、デフォルト値でコンパイル・コマンドを実行する現在のディレクトリが指定されます。
[-c]	コンパイルするとき、DECで指定されたパラメータのメニュー・フィールドの長さアクション・フィールドのイメージ(image)データ長を比較し、異なる場合は警告メッセージを出力します。
[-d]	デバッグ・モードでコンパイルします。コンパイルされたマップ・ファイルの構造体に保存された情報を出力します。
-v <i>mapfile</i>	コンパイルとは関係のないオプションであり、コンパイル済みのマップ・ファイルのバージョン情報を出力します。
[-h]	tsomapgenの使用方法を出力します。

- 入力項目

項目	説明
<i>menuname</i>	コンパイルするメニュー定義体の名前を指定します。 コンパイルされたマップ・ファイルは、{ <i>menuname</i> }.mapの形式で保存されます。

第4章 画面処理の応用

本章では、画面の入出力機能の応用について説明します。物理画面に対応する標準的な論理画面より大きい小さい論理画面を構成することが可能です。これにより、単純なデータの入出力だけではなく、画面の操作を行うことができます。

4.1. LSDA

IPFOPN/IPFOVSを処理するとき、論理画面の属性を定義するパラメータであるLSDA(Logical Screen Data Area)を利用して論理画面の属性を定義することができます。同パラメータを省略した場合には、標準的な論理画面の属性が適用されます。

LSDAの設定により、以下の機能を利用することができます。

- 論理画面を物理画面より大きく作成することができます。
- 画面分割を利用して、画面のレイアウトを自由に設計することができます。
- スクロールが可能な部分に対して、画面上に固定領域を設定することができます。
- 画面の移動幅を自由に設定することができます。

LSDAで設定した上記の機能は、IPFOPN/IPFOVSが一度呼び出されると、以降は変更することができないため、慎重に設定する必要があります。LSDAで設定する項目は、以下の12項目であり、各項目は、LSDAでそれぞれ2バイトの領域の占めます。

項目	説明
SIZE1	論理画面の行のサイズ
SIZE2	論理画面の列のサイズ
HARD1A	ハードウェアの表示領域（開始行）
HARD1B	ハードウェアの表示領域（最終行）
HARD2A	ハードウェアの表示領域（開始列）
HARD2B	ハードウェアの表示領域（最終列）
SCR1	画面の移動幅（行方向）
SCR2	画面の移動幅（列方向）
NOT1	画面の固定領域（行位置）
NOT2	画面の固定領域（列位置）
DISP1	物理画面との差（行方向）

項目	説明
DISP2	物理画面との差（列方向）

参考

LSDAを省略しない場合には、論理画面のサイズ、ハードウェアの表示領域を必ず指定します。それ以外の場合、ゼロを指定すると標準画面と同じ値に設定されます。

4.2. 画面分割

画面分割とは、物理画面に複数の論理画面を出力することです。LSDAを利用してそれぞれの画面サイズの属性を定義することができます。

4.2.1. 複数の論理画面

2つ以上の論理画面を順番に開くことで、物理画面を分割して論理画面を割り当てることができます。まず、1つの画面を開いた後、その上の一部の領域に次の画面を開いて上書きする方法で、複数の論理画面を出力することができます。

画面を分割する場合には、表示するそれぞれの論理画面のサイズをLSDAのハードウェアの表示領域(HARD1A、HARD1B、HARD2A、HARD2B)に指定します。

4.2.2. 分割された画面の識別

分割された画面は、論理画面の識別領域(LSIA)によって識別されます。したがって、複数の論理画面を使用するときは、複数のLSIAが必要です。すなわち、論理画面とLSIAは1対1で対応する必要があります。すでに開かれた論理画面に対応するLSIAを使用して別の論理画面を開くことはできませんが、終了された論理画面に対応するLSIAを再使用することは可能です。

4.3. 画面移動(スクロール)

物理画面のサイズに対応する標準的な論理画面の最大サイズは、横80×縦24に制限されます。論理画面のサイズを物理画面のサイズより大きく設定した場合、表示領域を超えた部分は、画面に一度に表示することができません。IPFでは、指定されたファンクション・キーの入力により、論理画面上の出力位置を移動させることが可能です。

4.3.1. 画面移動機能

物理画面のサイズより論理画面のサイズが大きい場合、物理画面は論理画面のすべての内容を表示することができません。このような場合、表示されなかった部分を表示する機能が画面移動機能です。

画面移動機能は、上・下・左・右の4方向に実行することができるため、この機能を利用すると、大きな論理画面も物理画面の表示領域を利用して入出力処理を行うことができます。LSDAのSCR1とSCR2に指定された数値の分だけ、それぞれの行方向と列方向へ画面移動を実行します。

4.3.2. 画面移動のファンクション・キー

IPFでは、以下のファンクション・キーを画面移動キーとして指定しています。

ファンクション・キー	説明
PF7、PF19	上方向
PF8、PF20	下方向
PF10、PF22	左方向
PF11、PF23	右方向

4.4. 画面固定

論理画面のサイズを物理画面のサイズより大きく設定した場合、画面移動機能を使用することができます。そのとき、全画面を移動するのではなく、一部の画面を固定することが可能です。

以下は、画面固定の規則です。

- 標準的な論理画面であるか、LSDAの画面固定領域の値がゼロの場合には、画面固定領域がありません。
- LSDAの画面固定領域のパラメータのうち、NOT1は1行からの行の数を意味しており、NOT2は1列からの列の数を意味します。

